

BHTTP/1: a binary framing for HTTP

Specification, version 1

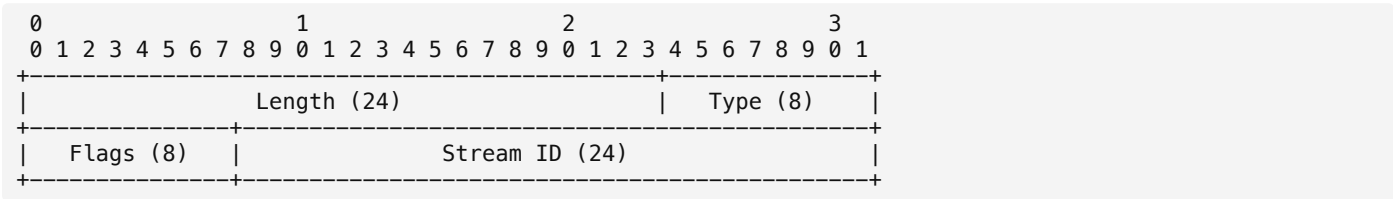
MUST, MUST NOT, SHOULD and MAY are used as in RFC 2119. Integers are unsigned and big-endian. Sizes are in bytes.

1. Connection

BHTTP/1 runs over one TCP connection (default port 9000). The server answers requests in order on the connection they came in on. A client MUST NOT open a second connection during one run (one set of requests). If the connection closes early, the client stops and reports an error rather than reconnecting. A client SHOULD wait for the end of a response before sending the next request. A server MUST still accept requests sent earlier (pipelining) and answer them in order. It reads each request completely before answering it. A server closes the connection only after a connection error (§6), when a request or response stops moving, or when no request has been in progress for a while (at least 10 s RECOMMENDED; unknown frames do not count as activity). It never closes with a fully received request unanswered, so a request with no response was not processed.

2. Frame header

Every frame is an 8-byte header followed by Length bytes of payload.



- Length: payload size, not counting the header (0 to 16 777 215).
- Type: 0x00 DATA, 0x01 HEADERS. Any other value is an unknown type (§3).
- Flags: 0x01 END_STREAM marks the last frame of a request or response. Senders set the other bits to 0 and receivers ignore them.
- Stream ID: the client numbers its requests on a connection, starting at 1. Each request MUST use a higher ID than the one before (gaps are allowed), and the response carries the same ID. Stream 0 stands for the connection itself: in v1 it carries only unknown frames and the connection-error message of §6. After using ID 16 777 215 the client MUST close the connection.

3. Unknown frame types

A receiver that reads a frame of unknown type MUST skip it: read and discard exactly Length payload bytes, then go on to the next frame. It MUST NOT reply to it, close the connection because of it, or let it change any request state, whatever its stream ID or flags (END_STREAM on an unknown frame ends nothing). This holds everywhere in the frame sequence, including between the frames of one request or response. Types 0xF0–0xFF will never be assigned, so senders can use them to test that peers skip; 0x02–0xEF are left for later versions. HTTP/2 has the same rule outside its header blocks (RFC 9113 §4.1, §5.5).

4. Messages

A request is one HEADERS frame and then zero or more DATA frames on the same stream, with END_STREAM on the last frame. A response has the same shape. A message with no body is a single HEADERS frame with END_STREAM set. The header block MUST fit in one HEADERS frame of at most 65 535 bytes; a receiver MUST accept any block within that size, however many fields it holds. Senders SHOULD keep DATA frames to 16 384 bytes or less; receivers MUST accept any Length. A body ends at END_STREAM. A response to HEAD is always a single HEADERS frame with END_STREAM. Any other response that carries content-length MUST carry exactly that many DATA bytes, and a client treats a difference as a protocol error. In v1 a server ignores a request's content-length. A sender that cannot finish a body (for example, a file read fails) MUST close the connection without sending END_STREAM, so the client can tell the body is incomplete.

5. Header block

A HEADERS payload is a sequence of fields with no count and no terminator; the frame's Length bounds it. In the grammar, tag is one byte and len is one or two bytes.

field	= tag [name] value	; name present only when tag = 0	
name	= len bytes	value = len bytes	
len	= 0xxxxxxx	; 0 to 127	
	1xxxxxxx xxxxxxxx	; 0 to 32 767 (15 bits, big-endian)	

Tag 0 means the name follows as a literal. Tags 1 to 10 name the fields of a plain GET exchange, listed below. Tags 11 to 255 are not defined in v1.

tag	name	tag	name	tag	name	tag	name	tag	name
1	:method	3	:status	5	user-agent	7	server	9	content-type
2	:path	4	host	6	accept	8	date	10	content-length

- Literal names are 1 to 32 767 lowercase HTTP token characters (RFC 9110 §5.6.2), so pseudo-headers (names starting with :) can only be sent by tag. Senders use the tag for any name in the table; receivers also accept the other table names as literals.
- Values are 0 to 32 767 bytes and may hold any byte except NUL, CR and LF. content-length is 1 to 18 ASCII digits. date uses the HTTP date format (RFC 9110 §5.6.7). Senders SHOULD use the one-byte len when it fits; receivers accept both forms.
- Pseudo-headers come before all other fields. :method, :path, :status, host and content-length appear at most once each, counted by name whether sent by tag or literal; other names may repeat.
- A request has one :method, a case-sensitive HTTP token, and one :path, starting with / and at most 1024 bytes as sent. It has no :status, and SHOULD have host.
- A response has one :status, three ASCII digits from 200 to 599, and no :method or :path. v1 has no 1xx responses.
- A block that breaks any of these rules, uses tags 11 to 255, or has a field whose length runs past the end of the payload is malformed.

6. Server

The server cuts :path at the first ? or #, decodes %XX escapes (%2F is a slash, %2e%2e is ..), refuses any .. segment and resolves the rest under its root, following symbolic links. The result must lie inside the root, have no component starting with a dot, and be a regular file; a directory or a trailing / means index.html. A found file gets 200 with content-type, content-length, server and date, then the file as DATA; errors get a short text/plain body; HEAD gets GET's headers and no DATA. Checks run in table order.

status	when
400	malformed header block (§5), block over 65 535 bytes, bad % escape (not two hex digits) or %00
405	a well-formed method other than GET or HEAD
403	a .. segment, a path resolving outside the root, or a file the server may not read
404	anything else that is not a regular file inside the root, including hidden (dot) files

These are stream errors: the response goes on the request's stream and the connection stays open. Any request body is read and discarded first. A connection error means the frame sequence cannot be trusted any more: HEADERS on stream 0 or with an ID not higher than the last; DATA with no request open or on a different stream; or a new HEADERS before the open request's END_STREAM. Requests completed before the bad frame are answered first. Then the server sends the connection-error message on stream 0 and closes: a HEADERS frame (:status 400, content-type, content-length) and then DATA holding the reason, the last frame with END_STREAM. If a frame is cut off by EOF the peer has gone, so the server just closes.

7. Client

The client matches responses to requests in order. A HEADERS frame on stream 0, at any point, starts a connection-error message; the client reads it and its DATA, reports the reason and stops. These are protocol errors, after which the client closes the connection: a frame for any stream other than the oldest unanswered request; DATA before HEADERS or after END_STREAM; a second HEADERS in one response; DATA in a response to HEAD; a malformed or oversized response header block; a content-length mismatch; a frame cut off by EOF; and no data for longer than the client is willing to wait.

8. Design notes

HTTP/2's header is Length 24, Type 8, Flags 8, then a reserved bit and a 31-bit Stream ID: 9 bytes. Its drafts used 8 bytes (a 16-bit Length in draft 04, 14 bits plus two reserved bits in draft 13) until draft 14 (2014) widened Length to 24 bits. Frames over 2¹⁴ bytes stay off unless the receiver allows them with SETTINGS_MAX_FRAME_SIZE, which also stops one stream holding up the others. Both ends start streams, client streams odd and server streams even, so the client has 2³⁰ IDs and the server 2³⁰ - 1 (0 is reserved). The top bit is reserved with no meaning (RFC 9113 §4.1); in SPDY, HTTP/2's predecessor, that bit marked control frames.

BHTTP/1 uses 24 / 8 / 8 / 24, or 8 bytes:

- Length 24. v1's own limits (16 KiB DATA, 64 KiB - 1 header blocks) fit in 16 bits; the extra byte is headroom for v2. The width cannot change later without breaking §3, because skipping depends on every version reading Length the same way.
- Type 8 and Flags 8. v1 uses two types and one flag. A v2 can add frame types freely (§3), but a v1 peer would ignore a new flag and reject a new tag, so those need the peer's agreement first (below).
- Stream ID 24. v1 answers strictly in order and has no server-started streams, so it needs no odd/even split or reserved bit; the ID only matches responses to requests. At 1000 requests a second, 2²⁴ IDs last 4.6 hours.

HPACK has a 61-entry static table of name-value pairs, a dynamic table, Huffman coding and prefix-coded integers.

BHTTP/1 takes two of its ideas: names from a fixed table (HPACK's "literal with indexed name") and length-prefixed literals. HEXDUMP.md's /hello.txt request without its extra header is 57 bytes; as HTTP/1.1 text it is 85.

There is no preface: a misdirected HTTP/1.1 GET / reads as an unknown frame of Length 0x474554, skipped until the idle limit closes the connection, a slow failure v1 accepts. To grow, a v2 client first sends a new frame type, say SETTINGS. A v2 server acknowledges it before its first response and a v1 server skips it, so a first response with no acknowledgement means stay with v1; no timer is needed.